Exercise Sheet 14

The Charter · two sessions

House rules for the project.

- First: `pip install -r requirements.txt`. Then create `my_work/charter.py`: your functions. It never prints and never names a file; everything it needs (a path, a connection, a DataFrame) comes in as an argument. Per exercise, a small `checkN.py` imports it and prints. In session 2 you add `report_console.py` and `report.py`.
- Run everything from the lesson folder: `python my_work/check1.py`. Because the check script lives next to `charter.py`, `import charter` just works.
- Use both crafts: **SQL** (`pd.read_sql` on `data/guild.db`) where the sheet says Vault, **pandas** on the CSVs where it says ledger. Every function gets a one-line docstring.
- Hand in at the end of **each** session: commit, push to `lesson-14`. Open the pull request after session 2.

The rules of the game: the Queen's questions

read first

The Royal Charter is renewed at Midsummer if the guild can show, properly, which quests are worth the crowns, which are too dangerous, and whether loot is shared fairly. The Privy Council has agreed what "properly" means:

QUESTION	MEASURED AS
1. How often do we win?	**Success** / all quests. **Fled is not a success**, even though it pays a quarter.
2. Does difficulty matter?	success rate and injuries per quest, per difficulty 1–5
3. Which monsters are worth it?	crowns **paid** per quest-day: `sum(paid) / sum(days)`, not the reward on the notice
4. Which regions?	quests, success rate and crowns paid per region, with its warden; **every** region, warden or not
5. Who gets the loot, and is it fair?	top heroes by loot; average difficulty and loot per trip by class; the leader's double share

Where the data lives: `data/guild.db` (the Vault: tables `heroes`, `quests`, `party`, `regions`) and the same tables as CSV files. `paid` = the reward on Success, 25% on Fled, 0 on Failed.

Session 1 · the numbers

Exercise 1 Open the Vault

15 min · `charter.py`, `check1.py`

> **RAVEN #1401 from Orrin of the Clockworks:** *"The Vault is yours. Start with the only question anyone ever asks me: how many of our quests actually end well? Dalvric used four hundred lines. Please use fewer."*

GIVEN (TOP OF CHARTER.PY)

```
"""charter.py: the Charter Report for Queen Aldara. Functions only."""
import sqlite3

import pandas as pd
```

(a) `open_vault(path)` returns `sqlite3.connect(path)`. `load_quests(path)` reads the CSV, adds `is_success` (`outcome == "Success"`) and returns the DataFrame.

(b) `outcome_counts(con)`: one SQL query with `GROUP BY outcome`, most quests first, via `pd.read_sql`.

(c) `success_rate(quests)`: the share of successes in %, as `float(round(..., 1))`.

(d) `check1.py` prints the number of quests in the ledger, the outcome table and the success rate.

EXPECTED OUTPUT

```
Quests in the ledger: 400
   outcome  quests
0  Success     259
1     Fled      82
2   Failed      59
Success rate: 64.8%
```

Exercise 2 Does difficulty matter?

15 min · `check2.py`

> **RAVEN #1402 from Captain Siv:** *"I grade every notice from one to five skulls. Do the skulls mean anything? Yes or no. And how many of my heroes come back limping?"*

(a) Write `by_difficulty(quests)`: group by `difficulty` with named aggregations: `quests` (count), `success_rate` (mean of `is_success`) and `injuries_per_quest` (mean of `injuries`).

(b) Turn the success rate into % with 1 decimal; round injuries to 2. Return the table.

(c) `check2.py` prints it.

EXPECTED OUTPUT

```
            quests  success_rate  injuries_per_quest
difficulty
1               34          88.2                0.12
2              110          74.5                0.22
3              128          62.5                0.65
4               88          56.8                1.08
5               40          42.5                1.78
```

Hint: Yes, Captain. From one skull to five, success halves and injuries go up fifteen-fold.

Exercise 3 Worth the crowns?

15 min · `check3.py`

> **RAVEN #1403 from Quartermistress Pell:** *"Everyone wants the Wyrm notice. It says three thousand crowns. I keep the accounts, and I have never once received three thousand crowns. What does each monster actually bring in, per day the heroes are away? P."*

(a) Write `per_day(quests)`: per monster, `quests` (count), `avg_reward` (mean, rounded to 0), `total_paid` (sum of `paid`) and `days` (sum).

(b) Add `paid_per_day` = total_paid / days, rounded to 1. Sort it best first and return it.

(c) `check3.py` prints the table and the best and worst monster.

EXPECTED OUTPUT

```
              quests  avg_reward  total_paid  days  paid_per_day
monster
Trolls            31      1309.0       20870   276          75.6
Giant Spiders     42       834.0       26060   363          71.8
Bandits           72       553.0       30080   558          53.9
Ghouls            61       722.0       27710   528          52.5
Wolves            64       349.0       16060   471          34.1
```

```
Goblins            81       313.0       18480   565          32.7
Wyrm                7      3130.0        1970    69          28.6
Giant Rats         42       145.0        4920   279          17.6
Best: Trolls, worst: Giant Rats
```

Hint: The biggest reward on the board, and second from the bottom. Seven Wyrm quests, not one won: a reward you never collect is worth nothing, however nicely it's written. That's why we measure `paid`, not `reward`.

Exercise 4 The regions and their wardens

15 min · `check4.py`

> **RAVEN #1404 from Orrin of the Clockworks:** *"The Vault has a table of regions and wardens. Thornwood isn't in it: the post has been vacant since the last warden met a bear. Thornwood still has quests, though. Lots. Don't lose it."*

(a) Write `by_region(con)`: one SQL query, `quests` **LEFT JOIN** `regions`, grouped by region, with `COALESCE(r.warden, '(none)') AS warden`, `COUNT(*) AS quests`, `ROUND(100.0 * SUM(q.outcome = 'Success') / COUNT(*), 1) AS success_rate` and `SUM(q.paid) AS paid`, ordered by paid, highest first.

(b) Return it with `set_index("region")`. `check4.py` prints it.

EXPECTED OUTPUT

```
                              warden  quests  success_rate   paid
region
Ironhills       Warden Hild Stonebridge      85          61.2  34730
Northern Wilds  Warden Corwin Northwind      75          64.0  33600
Thornwood                       (none)      90          71.1  32780
Saltmarsh        Warden Nerys Saltmarsh      82          69.5  26890
Greyfen               Warden Alder Pike      68          55.9  18150
```

Hint: In SQLite, `q.outcome = 'Success'` is 1 or 0, so `SUM` counts the successes. Try a plain `JOIN` once and watch the guild's busiest region vanish.

End of session 1. Commit (`git add . && git commit -m "Charter: session 1, the numbers"`) and push. Tell Lars the one number that surprised you most.

Session 2 · the report

Exercise 5 Who gets the loot, and is it fair?

20 min · `check5.py`

> **RAVEN #1405 from Queen Aldara:** *"The Bards have written us a song complaining that they are never sent anywhere interesting. It rhymes, which is suspicious. Who earns the loot, and is it shared out fairly? A."*

(a) `leaderboard(con, n=5)`, in SQL: `party JOIN heroes`, grouped by hero, with `name, class, level`, `COUNT(*) AS quests` and `ROUND(SUM(p.loot_share), 2) AS loot`, most loot first, `LIMIT ?` with `params=(n,)`.

(b) `by_class(party, heroes, quests)`, in pandas: merge party with heroes and quests; per class `trips`, `avg_difficulty` and `loot_per_trip` (mean of `loot_share`); round to 2, sort by avg_difficulty.

(c) `leader_take(party)`: keep rows with `loot_share > 0` on quests with more than one hero (`groupby("quest_id")["hero_id"].transform("count") > 1`); the leader's share is the biggest per quest. Return the leaders' total as % of all those shares, 1 decimal, plain float.

EXPECTED OUTPUT

```
                name    class  level  quests     loot
0    Jasper Ironfoot  Warrior     17      23  4428.68
1  Benedek Saltmarsh  Warrior     15      18  3833.34
2   Silas Goldenhand  Warrior     14      17  3502.33
3       Wren Ashdown   Ranger     14      20  3460.14
4   Oriel Quickwater  Warrior     11      21  3228.82

         trips  avg_difficulty  loot_per_trip
class
Bard        84            2.95         111.79
Cleric     131            3.24         110.27
Rogue      176            3.32         110.94
Mage       224            3.39         119.33
Ranger     215            3.46         126.46
Warrior    364            3.49         134.26

Leaders' share of the loot on shared quests: 47.2%
```

Hint: The Bards have a point: they get the easiest quests, while Warriors get the hardest ones and the biggest shares. And the leader (the highest-level hero) takes a double share, so on shared quests nearly half of all loot goes to one person per party. Fair? That's for the Queen.

Exercise 6 The console report

20 min · `report_console.py`

> **RAVEN #1406 from Queen Aldara:** *"One sentence a queen can say out loud at Midsummer without looking at a scroll. Then the detail underneath, for the councillors who like detail. There are always some."*

(a) In `charter.py`, write `one_sentence(quests)`: take `per_day(quests)`; the best monster is the first row, the "glamorous" one is `table["avg_reward"].idxmax()`. Return exactly the last line below.

(b) Write `report_console.py`: open the Vault, load the four tables, call each function once, and print the report below. The title is underlined with `"=" * len(title)`; the outcome summary is built from the `outcome_counts` table with `", ".join(...)`. Each table is printed after an empty line.

EXPECTED OUTPUT

```
THE CHARTER REPORT: ADVENTURERS' GUILD OF THORNMERE
==================================================
Quests in the ledger:   400 (259 Success, 82 Fled, 59 Failed)
Success rate:           64.8%
Best per quest-day:     75.6 cr (Trolls)
Worst per quest-day:    17.6 cr (Giant Rats)
Leaders' share of loot: 47.2%
```

Then the five tables you already know, each after an empty line, in this order: difficulty (Ex. 2), monsters (Ex. 3), regions (Ex. 4), the leaderboard and the classes (Ex. 5). They must match those exercises exactly. Then an empty line and:

```
The guild wins 65% of its quests, and Trolls pay best at 76 crowns a day; the Wyrm, for all
its 3,130-crown rewards, earns just 29.
```

Exercise 7 The chart and the workbook

25 min · `report.py`

> **RAVEN #1407 from Quartermistress Pell:** *"And now the version for the throne room: one picture of crowns per day, and a workbook with a tab per question, so the councillors can fiddle in the carriage home. Then we close the Grimoire. P."*

(a) Copy `report_console.py` to `report.py`. Add the matplotlib and openpyxl imports from lesson 13 and `OUT = Path("my_work/output")`, created with `mkdir`.

(b) Chart: `plt.barh` of `paid_per_day`, sorted smallest to largest, every bar navy except the Wyrm in coral (build a list of colours with a list comprehension). Title, x-label, save as `OUT / "crowns_per_day.png"`.

(c) Workbook `OUT / "charter_report.xlsx"`: a **Summary** sheet (two columns `measure` / `value`, ending with the Queen's sentence, `index=False`), then **Difficulty**, **Monsters**, **Regions**, **Heroes** (`index=False`), **Classes**.

(d) Re-open it; `for ws in wb.worksheets:` bold white on navy header, column A width 30, B–F width 16, freeze `A2`. Crown columns get `'#,##0.00 "cr"'`: Monsters C, D, F; Regions E; Heroes E; Classes D. Finally `wb.create_sheet("Chart")` with the PNG at B2. Save, and print the two lines below.

EXPECTED OUTPUT

Everything from Exercise 6, then an empty line and:

```
Sheets: ['Summary', 'Difficulty', 'Monsters', 'Regions', 'Heroes', 'Classes', 'Chart']
Saved my_work/output/charter_report.xlsx and my_work/output/crowns_per_day.png
```

Hint: A dict saves a lot of `if`s: `crowns = {"Monsters": "CDF", "Regions": "E", ...}`, then `for col in crowns.get(ws.title, ""):` and `for cell in ws[col][1:]:`. Open the workbook and the PNG. That's the report.

Bug hunt Pages from the Grimoire, page 14

15 min · `bughunt/grimoire_bug.py`

The last page. Before he sailed west, Dalvric copied the Grimoire's Charter incantation into Python. For years it has told the Privy Council that the guild wins 85% of its quests and should hunt nothing but wyrms. Your report disagrees. The Queen wants to know who's right. **Three** bugs: one crashes (a region has gone missing), one crowns the wrong monster, one counts running away as winning.

EXPECTED OUTPUT ONCE FIXED

```
THE GRIMOIRE'S CHARTER FIGURES, CHECKED
Success rate: 64.8%
Best monster per quest-day: Trolls (75.6 cr)
Thornwood: 90 quests, warden (none)
The Grimoire agrees with the Keeper. The Grimoire may now close.
```

One commit per bug. Then open the pull request, title: **"The Charter"**. In the description, paste the Queen's sentence and the one thing you'd tell the Privy Council that the numbers don't say.

More riddles. Five more torn pages wait in `bughunt/riddles/`, one bug each (the last one has two), getting harder as you go. Optional: for fast finishers, or a quiet evening.